

MEMORANDUM

RM-3588-PR

MAY 1963

THE HEURISTIC COMPILER

Herbert A. Simon

PREPARED FOR:

UNITED STATES AIR FORCE PROJECT RAND

The **RAND** *Corporation*
SANTA MONICA • CALIFORNIA

MEMORANDUM

RM-3588-PR

MAY 1963

THE HEURISTIC COMPILER

Herbert A. Simon

This research is sponsored by the United States Air Force under Project RAND—contract No. AF 49(638)-700 monitored by the Directorate of Development Planning, Deputy Chief of Staff, Research and Development, Hq USAF. Views or conclusions contained in this Memorandum should not be interpreted as representing the official opinion or policy of the United States Air Force.

The **RAND** *Corporation*

1700 MAIN ST. • SANTA MONICA • CALIFORNIA

PREFACE

The research reported in this Memorandum, rather than being aimed at the construction of a specific compiler (a computer program for translating instructions from a language convenient to the programmer to the machine language), is directed towards deepening our understanding of the kinds of problem-solving activity that are involved in computer programming, and the kinds of language and representational means that are needed to produce more sophisticated compilers. The Memorandum takes the form, therefore, of a series of illustrative problems of compiler design, with proposals, worked out in some detail, for their solution.

There has been, in the past decade, enormous progress in the development of higher-level programming languages for instructing computers. Through the invention of algebraic compilers, like FORTRAN, IT, and ALGOL, data processing languages like COBOL, and list-processing languages like IPL, LISP, and COMIT, the labor of programming has been reduced several orders of magnitude. Yet, when we are faced with the complexities of modern command and control systems, and the programming problems they present, we recognize that the progress made to date is not nearly enough. Programming a computer to perform a complex task is still very much more intricate and tedious than instructing an intelligent and trained human being for that task.

Most visibly, the human--if, to repeat, he is intelligent and knowledgeable--does not have the literalness of mind that is so characteristic of the computer, and so exasperating in our interaction with it. From his own store of knowledge, he supplies facts that we neglect to give him; given statements of objectives in broad functional terms, he applies his problem-solving powers to filling in the detail of method; confronted with the vagueness and informality of natural language, he interprets meaning and intent.

The experiments reported in this Memorandum aim at further bridging the gap between the explicitness of existing computer programming languages and the freedom and flexibility of human communication. The work was motivated by the belief that until we bridge that gap, we shall not be able to harness effectively the powers made available to us by modern digital computers and apply them to the vast systems (e.g., command and control) that are becoming such a central feature of our military and civilian technology.

This work is part of The RAND Corporation's continuing program of research in the area of complex information processing, under U.S. Air Force Project RAND. The author, a RAND consultant, is a member of the faculty at Carnegie Institute of Technology. The Memorandum is directed primarily to systems programmers who are faced with problems

of compiler design, with the hope that its proposals will suggest means for increasing the power, generality, and flexibility of compiling systems.

SUMMARY

Two major themes run through the topics discussed in this Memorandum. The first thesis is that more of the programming burden can be shifted from programmer to computer if the computer is given some problem-solving powers. In previous works it has been shown how a computer program, the General Problem-Solver, can simulate the kinds of means-end analysis that humans use to solve problems.* Part I of the present Memorandum shows how a compiler can be designed that makes use of heuristic problem-solving techniques like those incorporated in the General Problem-Solver (GPS). Such a scheme permits a desired program to be specified in general terms, with the compiler using means-end analysis and selective trial-and-error search to work out the exact "how" of it.

The second main theme is that if we are to have flexibility in a compiler language commensurate with the flexibility of natural language, we must first gain an

*The General Problem-Solver has been described, and its behavior analyzed, in several RAND publications by A. Newell, J. C. Shaw, and H. A. Simon: Report on a General Problem-Solving Program, P-1584, February 9, 1959; The Simulation of Human Thought, P-1734, June 22, 1959; A Variety of Intelligent Learning in a General Problem Solver, P-1742, July 6, 1959; GPS: A Program that Simulates Human Thought, P-2257, April 10, 1961; Computer Simulation of Human Thinking, P-2276, April 20, 1961; Computer Simulation of Human Thinking and Problem Solving, P-2312, May 29, 1961.

Also see, A Guide to the General Problem-Solver Program GPS-2-2, Allen Newell, RM-3337-PR, February 1963.

understanding of the ways in which meanings are represented in natural language, and then devise representations of corresponding power (and ambiguity) for compiling languages. Parts II and III are devoted primarily to questions of language and representation. They provide a number of suggestions for increasing the generality and flexibility of compiler languages.

The boundaries between the three parts are largely chronological. Part I represents work completed during the winter of 1960-61; Part II, work done during the spring of 1961; and Part III, work done since the summer of 1961, particularly during the summer and autumn of 1962. A program listing for the main portion of the compiling scheme, described in Sections I-V of Part I, is given in Appendix A. A program listing for the annexing scheme, described in Sec. IX of Part III, is given in Appendix B.

ACKNOWLEDGMENT

I am greatly indebted to H. S. Kelly, of The RAND Corporation, for numerous discussions of the work described in this paper, and for help in solving many of the problems that arose.

CONTENTS

PREFACE	iii
SUMMARY	vii
ACKNOWLEDGMENT	ix
 <u>PART I: A PROBLEM-SOLVING COMPILER</u>	 1
I. THEORY OF PROBLEM SOLVING	2
II. PROGRAM WRITING AS PROBLEM SOLVING	5
III. OUTLINE OF A HEURISTIC CODER FOR IPL-V ...	6
The SDSC Compiler	7
The DSCN Compiler	12
The General Compiler	15
IV. RELATION OF THE HEURISTIC COMPILER TO GPS.	20
V. FLOW DIAGRAMS	21
 <u>PART II: GENERAL IMPLICATIONS FOR REPRESENTATIONS.</u>	 27
VI. LANGUAGE AND REPRESENTATIONS IN THE COMPILER	28
Descriptive Names	28
State Descriptions	31
VII. THE DESIGN OF REPRESENTATIONS	34
 <u>PART III: EXPERIMENTS WITH REPRESENTATIONS</u>	 41
VIII. GENERALIZED PROCESSES	44
"FIND" Processes	44
"SORT" Processes	48
Recursive Functional Languages	51
IX. DEFINITE DESCRIPTIONS	53
Syntactic Characterization	53
Semantic Characterization	54
Annexing Descriptive Information to an Information Store	55
X. COMPILATION OF ROUTINES FROM PARTIAL DESCRIPTIONS	60

Appendix

A.	PROGRAM LISTING OF THE HEURISTIC COMPILER	65
B.	PROGRAM LISTING OF AN INFORMATION- ANNEXING SCHEME	118

PART I

A PROBLEM-SOLVING COMPILER

In this first part, we describe a compiler that makes use of heuristic problem-solving techniques like those incorporated in the General Problem-Solver (GPS).^{*} Section I provides a brief introduction to the theory of problem solving and the structure of GPS. Section II shows how programming can be interpreted as a problem-solving activity, within this framework. Section III describes the main components of the compiler, and Sec. IV indicates how the compiler could be incorporated as a set of subroutines to GPS. Section V describes an extension of the compiler to handle flow diagrams. A complete program listing for the compiler is given in Appendix A.

^{*}Op. cit., p. vii.

I. THEORY OF PROBLEM SOLVING

The motivation for the Heuristic Compiler is supplied by a theory of problem solving that also provides the basic framework for the General Problem-Solver. By a problem, we mean a situation of the following kind:

1. We are given a (partial) description of a present situation and a desired situation. These situations are described in a language that we may call the state language. The state language is sufficiently rich to permit us to describe situations (we shall call such descriptions objects), and to describe differences between pairs of situations.
2. We are given a list of operators that can be applied to situations to transform them into new situations. Operators are named in a language that we may call the process language. Any sequence of operators named in the process language, also names an operator--the compound operator that consists in applying, successively, the elementary operators belonging to the sequence.
3. A problem solution is a (compound) operator in the process language that will transform the object describing the present situation into the object describing the desired situation.

EXAMPLE: We take as the objects in the state language the integers, 1, 2, We take as the elementary operator the successor operation, which we shall designate as ' in the process language. Then '' and '''' are examples of compound operators. Consider the problem of transforming the present object 5 into the goal object 8. The solution is the operator '', for $5'' = 8$. More generally, '' is the operator that removes the difference +3 between any two objects, x and y; for if $y - x = +3$, then $x'' = y$. Here +3 is a difference in the state language; '' is the operator relevant to that difference in the process language. We may construct a table of connections to associate with each difference the operator or operators relevant to it.

With this formulation of the concept of "problem," many techniques of problem solving can be subsumed under the following general paradigm:

MEANS-END ANALYSIS. Given the present and desired objects, find a difference between them. Next find an operator relevant to the difference; determine if the operator can be applied to the present object. If so, apply it. (If not, describe the objects to which it would apply and transform the present object into an object of that kind--a new "desired object.") Take the new object thus obtained as the present object and repeat the process.

The General Problem-Solver is a program that uses this scheme of means-end analysis for attempting the solution of any problem that can be cast into the form described.*

*This is a bare-bone description of GPS, but it will suffice for our purposes.

II. PROGRAM WRITING AS PROBLEM SOLVING

The task of proving a theorem can be formulated as a problem for GPS. The desired object is the theorem to be proved. The present object is the set of axioms and already-proved theorems. The operators are the legitimate processes for transforming a subset of axioms and/or theorems into a new theorem. We have a proof when we have a sequence of operators that transforms the present object into the desired object. (What we call a proof here is usually regarded as the justification for the proof steps; the proof, as usually written out, consists of the sequence of successive transformations of the axioms and given theorems.)

The sequence of operators that constitutes a proof can also be interpreted as a program that generates the desired object from the given object; for if we apply the operators of the proof, in sequence, to the present object (the axioms and previous theorems), we obtain precisely the desired object--the theorem to be proved. Thus, a theorem-proving system can be regarded, at least formally, as a program-writing system. Conversely, if we can formulate a programming goal as a difference between a present and a desired object, we can presumably use the same processes, which in the other context will generate the proof of a theorem, to generate a program.

III. OUTLINE OF A HEURISTIC CODER FOR IPL-V

In the remainder of Part I, I shall describe a number of routines for compiling programs in Information Processing Language V (IPL-V), an interpretive list-processing language.* What is common to all of these compiling procedures is that they embody the problem-solving notions discussed in the preceding paragraphs. That is, each of the compiling routines accepts the task of writing programs in IPL-V on the basis of certain information provided to it. The task is accomplished by the application of the means-end analysis that has been described. The several compiling routines differ with respect to their methods of formulating or representing the problem--that is, each operates with a different state language. At present, there are three compiling routines:

1. SDSC Compiler (State Description Compiler) [U140].**

This routine takes as its input a description (SDSC) of the contents of the relevant computer cells before and after the routine to be compiled has been executed. It produces an IPL-V routine that will transform the input state description into the output state description.

*See Newell, Allen (Ed.), Information Processing Language V Manual, Prentice-Hall, Englewood Cliffs, N.J., 1961.

**Expressions in square brackets are names of the corresponding routines, data list structures, and symbols in the compiler program.

2. DSCN Compiler (Descriptive Name Compiler) [U134].

This routine takes as its input a verbal definition (in the form of an imperative sentence) of the routine to be compiled. It produces an IPL-V routine that is the translation, in the interpretive language, of that definition.

3. General Compiler [U135]. This is an executive routine that can use the SDSC Compiler, the DSCN Compiler, and others as subroutines. It takes as its input information about the routine to be compiled; the information can be stated in any one of several representations (e.g., those appropriate to SDSC or DSCN). The routine then selects subroutines that can use this information to produce the desired IPL-V code.

From a logical standpoint, we could describe the Heuristic Coder as a single program whose executive routine is the General Compiler, and which contains the SDSC Compiler and the DSCN Compiler as subroutines. For clarity of exposition, it will be better to describe the two parts first as independent programs, and then show how they are imbedded in the General Compiler.

THE SDSC COMPILER

A computer routine can be defined by specifying the changes it produces in the contents of the storage location it affects, or, what amounts to almost the same thing, by specifying the before and after conditions of

these storage registers. A definition of this kind is not, of course, univocal, for programming is a synthetic, not an analytic task; there will generally be many programs (not all equally efficient or elegant) that will do the same work. As presently constituted, the SDSC Compiler attempts to find some one routine to accomplish a given task.

EXAMPLE: In IPL-V there is a process, PUT SYMBOL J3 IN MEMORY LOCATION H5, which affects a single memory location, H5. This process [X105] happens to have the name J3; its state description (SDSC) is the following:

BEFORE J3 is executed, cell H5 contains a symbol, call it S1, followed by an indeterminate list of symbols, R0 (call this the pushdown list associated with H5).

AFTER J3 has been executed, cell H5 contains the symbol J3, followed by the same list of symbols, R0, as before. The token of symbol S1 that was previously in H5 has been destroyed.

Notice that it is implicit in this SDSC definition of J3 that the content of no cell other than H5 has been altered by the routine. We can represent the SDSC [the value of attribute X24 of the routine] diagrammatically as follows:

SDSC of J3

Affected Cells

H5

Input
Output

S1,R0
J3,R0.

Generalizing, the SDSC of a routine consists of a list of affected cells [attribute X71 of X24]. For each affected cell on the list, the SDSC specifies its input state [attribute X75 of X71] and its output state [attribute X76 of X71].

To compile the IPL-V code (JDEF) for J3, the SDSC Compiler [U140] proceeds as follows:

1. It matches [X7] the input states with the output states of the affected cells until it finds a difference. In the example cited, the difference between the input and output states of H5 may be called a replacement in H5.

2. It searches [U150] a table of connections [X90] that associates with each difference a list of operators (compiled IPL-V routines) that are relevant to that difference. In the example, the table of connections would contain, associated with the replacement difference [X83], the IPL-V routine P2(C) [X106]. P2(C) replaces the symbol in cell C, a variable, with the symbol in cell H0. Thus, P2(C) has the following SDSC [see local 95 of X106]:

SDSC of P2(C)

	<u>Affected Cells</u>	
	<u>H0</u>	<u>C</u>
Input	S2,R0	S1,R0
Output	R0	S2,R0.

3. It tentatively applies [U141] the relevant operator it has found to the input state of the SDSC to be compiled, and determines the resulting output state. In applying the operator, it makes appropriate substitutions for the variables in the operator [U153, U154]. Thus, applying P2(C) to the input of J3, we find, by matching, that we should set $C = H5$ and $S2 = J3$, giving:

SDSC of P2(H5)

	<u>H0</u>	<u>H5</u>
Input	J3,R0	S1,R0
Output	R0	J3,R0.

4. The application of the operator creates two new subproblems: Let I_a be the input state of the routine to be compiled, O_a its output state, I_b the input state of the operator, and O_b its output state. The original problem was to transform I_a into O_a . The new problems are: (1) to transform I_a into I_b (i.e., to establish the input conditions for application of the operator), and (2) to transform O_b into O_a (i.e., to transform the output state of the operator into the desired output state of the routine to be compiled). Either of these new

problems may reduce to the identity transformation, in which case that part of the problem is solved. If this reduction does not occur, then the same steps, 1, 2, 3, are applied [recursion of U140] to the new subproblem.

In the example at hand, 0_b is identical with 0_a ; hence, the remaining subproblem is to transform I_a into I_b , that is, to compile a routine with SDSC:

H0
S2,R0
J3,S2,R0.

The repetition of step 1 for this subproblem discovers a new difference, an addition [X82] to H0. Step 2 finds the relevant operator, process P1(S) [X107], which adds the symbol S to the symbol list in H0. Applying, in step 3, the operator P1(J3), the input state of H0: S2,R0, is transformed into the output state, J3,S2,R0. Hence, the solution to the original problem of compiling J3 is obtained by the sequence, P1(J3), P2(H5), or, in the usual IPL-V format [X22 of the routine]:

J3 10 J3
 20 H5 0.

We see that for the SDSC compiler to operate, it must be provided with a set of differences and matching tests for noticing differences, a set of already-compiled operators, and a table of connections between differences and operators. Further, when it has compiled a new routine,

the compiler can annex this routine to its set of available operators and use it in compiling subsequent routines.

THE DSCN COMPILER

Let us now consider an alternative compiling scheme for the same routine, J3. Instead of specifying the before and after condition of the computer cells, we define the routine [X20 of the routine] in terms of the function it performs: REPLACE THE SYMBOL IN H5 BY J3. (This definition (DSCN) resembles more closely than the previous one the manner in which routines are defined for "conventional" compilers like FORTRAN or LISP. What distinguishes the present scheme from these is the use of heuristic means-end analysis for working from the definition to the compiled routine.)

The first step in the DSCN Compiler [U134] is to search a list of available (compiled) routines to find one whose DSCN is as similar as possible to the DSCN of the routine to be compiled. In the case at hand, we would find the routine P2(C): REPLACE THE SYMBOL IN C BY (H0).

At the second step [U130], means-end analysis is performed to transform the compiled routine that has been found into the new routine. The transformations are performed on the DSCN's. Thus, in the present example, there are two differences between P2(C) and J3. The

former refers to the cell C, the latter to H5; the former refers to the symbol that is contained in cell H0, the latter to the symbol J3.

The compiler notices these differences (in a sequence), and searches for an operator relevant to removing the differences. In this case, C can be transformed to H5 by a substitution operator. (H0) can be changed to J3 by an addition operator (MAKE (H0)=J3 BY ADDITION). The application of these operators to the DSCN of P2(C) would compile the desired routine in the following stages:

P2(C)	REPLACE THE SYMBOL IN C BY (H0).
Apply <u>substitution</u>	REPLACE THE SYMBOL IN H5 BY (H0).
Apply <u>addition</u>	REPLACE THE SYMBOL IN H5 BY J3.

The resulting program in this case is identical with that obtained by the SDSC Compiler.

A somewhat more complex routine compiled by the DSCN Compiler is:

J13: INSERT (1) AT THE END OF (THE VALUE OF ATTRIBUTE
(0) OF (2)).

The list of available IPL routines includes:

J65: INSERT (0) AT THE END OF (1).

The differences between J65 and J13 are in their arguments. J65 has the argument (0), where J13 has the argument (1); J65 has the argument (1), where J13 has the argument (THE VALUE OF ATTRIBUTE (0) OF (2)). Since

it is not easy in IPL-V to rearrange arguments that are located in the pushdown list of the communication cell, H0, the compiler facilitates matters by incorporating in the compiled routine an algorithm that moves the inputs of the routine to be compiled into known working storage locations, then puts these inputs back into H0 in the order in which they are needed for the subprocesses. That is, the compiler first transforms J13 into another routine, call it K13, which it then compiles. The DSCN of K13 is:

K13: INSERT 1W1 AT THE END OF THE VALUE OF
ATTRIBUTE 1W0 OF 1W2.

The code for J13 may be written as:

J13 J52
 K13
 J32 0 .

Now K13 is to be compiled with the aid of J65. Comparing the corresponding arguments of the two routines, we see that this involves finding the value of attribute 1W0 of 1W2, placing this value in H0, bringing 1W1 into H0, and then performing J65. That is to say, K13 will have the general form:

K13 FIND V(1W0,1W2)
 11W0
 J65.

In the list of available routines, the compiler finds:

J10: FIND THE VALUE OF ATTRIBUTE (0) OF (1),

which may be abbreviated, FIND V((0),(1)). Comparing the arguments of J10 with V(1W0,1W2), we see that (1) must

be set equal to 1W2 and (0) to 1W0. Hence, V(1W0,1W2)
is equivalent to

11W2
11W0
J10.

Hence, the complete code for K13 is:

11W2
11W0
J10
11W1
J65,

and the complete code for J13 is simply:

J13 J52
 11W2
 11W0
 J10
 11W1
 J65
 J32 0 .

THE GENERAL COMPILER

The General Compiler [U135] is an executive routine whose task is to compile a routine from information in any of the forms already discussed (SDSC and DSCN) or in other forms that may be described. It takes as its input the name of the routine to be compiled. Associated with this name (on its description list) is the information to be used in the compilation. More formally:

A routine is a description list containing values of some subset of the following attributes:

1. IPLN--IPL name [X25]. The value of this attribute is a description list that names a region

[X33] and a location [X34] in the region;
e.g., J60, R149, J3.

2. JDEF--IPL-V definition [X22]. The value of this attribute is a list of IPL-V instructions, each in the form of a description list describing [attributes X40-X46] the corresponding IPL-V word, that defines an IPL-V routine with the specified name. For example, the routine with IPLN J3 might have the following JDEF:

```
J3      10J3
        20H5      0 .
```

3. DSCN--Descriptive name [X20]. The value of this attribute is an imperative sentence (encoded as a list structure) that describes [with attribute X30,X31] the process defined by the JDEF. For example, the routine with IPLN J3 has, as already explained, the DSCN:

REPLACE THE SYMBOL IN H5 BY J3.

4. SDSC--State description [X240]. The value of this attribute is a list structure that describes the state of the IPL computer before and after the routine in question has been executed. Only changes are mentioned explicitly. Thus the SDSC of J3 is:

H5: S1,R0. J3,R0.

5. FLWD--Flow diagram [X267]. The value of this attribute is a list structure that gives the flow diagram corresponding to the JDEF. This list structure will be described in more detail later.
6. ASOJ--Associated J definition [X23]. The value of this attribute is the IPL name of a routine associated, in a manner to be described later, with a given routine.

A compiled routine is a routine that has a JDEF. Now we can state the problem of compiling a routine as follows: Given a routine without a JDEF (the present object), find the corresponding routine with a JDEF (the goal object). "Corresponding" means that the compiled routine has the same SDSC or DSCN as the given routine. Figure 1 presents the flow diagram of a compiler [U135] that uses means-end analysis to accomplish this compilation.

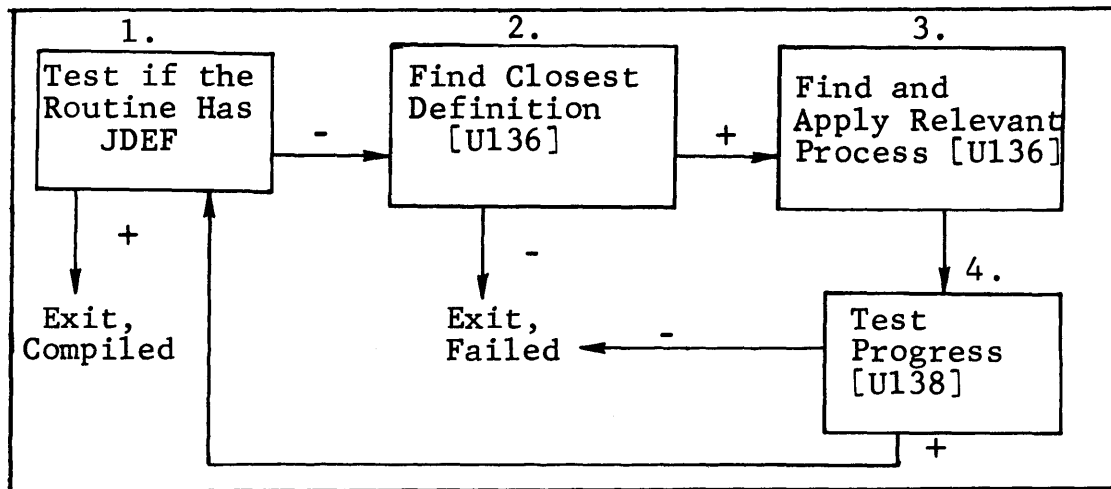


Fig. 1--Flow Diagram for: COMPILE ROUTINE R1

Let us translate this flow diagram into the language of means-end analysis.

1. Test if the routine has JDEF. This test determines whether the present object has the characteristics of the desired object. If so, the compilation is complete.
2. Find the closest definition [U136]. This process corresponds to finding a difference between the present and desired objects. However, we generalize this notion to mean look for a characteristic of the present object that will suggest a relevant operator. If the object possesses a DSCN, then an attempt could be made to compile the JDEF from the DSCN; if it possesses a SDSC, an attempt could be made to compile the JDEF from the SDSC. The attributes that the routine could possess are listed in an order that reflects the relative ease of compiling a JDEF from them. The process then finds the first attribute on this list that the routine to be compiled possesses. In the present form of the compiler, it is assumed that it is easier to compile from a DSCN than from a SDSC; hence, the attributes are listed in this order. If the routine possesses no attribute that can be used as a basis for compilation, the compiler reports a failure.

3. Find and apply the relevant process [U137]. The input to this process is the "closest definition" that has just been found. A table of connections is searched to find a process that is relevant to compiling the JDEF from the closest definition. If one is found, it is applied (in a manner to be described later).
4. Test progress [U138]. If the operator has been applied successfully, the routine will now possess at least one attribute (a JDEF or another) it didn't previously possess. If the progress test detects that it now has a definition closer to the JDEF than any it had previously, it initiates a new compilation cycle; if not, it reports a failure and quits.

The present list of "closest definitions" is very short, consisting only of DSCN and SDSC. The present table of connections [X198] is also brief:

1. If the routine possesses a DSCN, apply the operator, COMPILE JDEF FROM DSCN.
2. If the routine possesses only a SDSC, apply the operator, COMPILE JDEF FROM SDSC.

IV. RELATION OF THE HEURISTIC COMPILER TO GPS

Since each of the major components of the Heuristic Compiler is a system of means-end analysis, each of these components can be viewed as a rudimentary GPS. It should therefore be feasible, by modifying the top-level programs, to bring the Heuristic Compiler into a form that would allow its problem-solving processes to be governed by GPS. The programs for detecting differences, the tables of connections, and the operators would provide definitions of task environments for GPS. To accomplish this, GPS would have to be arranged so that a subproblem could be attached by applying GPS to a new task environment. That is, GPS would first be applied to the task environment of the General Compiler; applying an operator in this environment would consist in applying GPS to the task environment of the DSCN's or the SDSC's, as the case may be.

V. FLOW DIAGRAMS

Up to this point, we have considered only very simple programs, requiring no branches or loops. Each program is a list of instructions; each instruction is an IPL word represented as a description list with the attributes type [X40], name [X41], sign [X42], P [X43], Q [X44], symbol [X45], and link [X46].

To represent a program with branches and loops, we divide the program into segments. Each entry point to a loop (an instruction with a local name) begins a new segment; each branch instruction (P = 7) ends a segment. Each segment has the same attributes as an IPL word--specifically: name, P, symbol, link, and an additional attribute [X22], whose value is the list of IPL instructions for the segment. The name of the first instruction of the segment is assigned as the name of the segment; if the segment ends in a branch instruction, it is assigned P=7, and its symbol and link are set equal to the symbol and link of the branch instruction. If the segment does not end in a branch, it is assigned P = 0 and SYMB = 0, and its LINK is set equal to the link of its last instruction. Under these conventions, the list of segments is a flow diagram of the routine with the detail of the routine segments appended.

To illustrate the format of a flow diagram, we show below the code for IPL routine J77, followed by its flow diagram. The DSCN of J77 is: TEST IF THERE IS A SYMBOL EQUAL TO (0) ON LIST (1).

IPL-V CODE FOR J77

J77	J50	90	<u>Segment I:</u> Put (0) in W0.
90	J60		<u>Segment II:</u> Find next
	7091	92	location on list (1).
92	12H0		<u>Segment III:</u> Test if
	11W0		symbol at location is
	J2		equal to (0).
	7090	91	
91	30H0		<u>Segment IV:</u> Clean up
	J30	0	and exit.

FLOW DIAGRAM FOR J77

J77	0	90
90	7 91	92
92	7 90	91
91	0	0

From the description of the flow diagram, it is easy to provide a program [U139] that will construct a flow diagram from an IPL routine, and a program [U133] that will compile an IPL routine from the flow diagram and appended code segments. In this way the task of compiling an IPL routine is reduced to the problem of compiling its flow diagram, and compiling the code for each of the segments of the flow diagram.

The program for compiling such a routine from its DSCN has not yet been written, but examination of the structure of the routine itself shows what is involved. The test involves a quantifier--whether there exists a symbol with a certain property on a particular list of symbols. In IPL-V, such existence tests are performed by means of a loop or a generator; the members of the set in question are produced one by one and tested for their possession of the property. If a test result is positive, the process stops, and the signal, H5, is set plus. If the set is exhausted, the signal, H5, is set minus. Thus, a standard flow diagram can be used for all routines of this kind:

- A Perform required setup.
- B Locate another member of set
(if none, exit, via D).
- C Perform test on member
(if it succeeds, exit, via E;
if it fails, return to B).
- D Exit with signal minus.
- E Exit with signal plus.

Except for the provision of two distinct exits, this is identical with the flow diagram previously shown for J77 (Set A = J77, B = 90, C = 92, D = E = 91). Now, we can compile for each segment of the flow diagram a routine that corresponds to the DSCN of that segment. For example, FIND ANOTHER MEMBER OF (1) becomes J60 (after appropriate recognition of the changed location of (1)); PERFORM TEST ON MEMBER becomes:

12H0
11W0
J2 .

The only complications lie in moving the inputs for the various processes (J60 and J2) in appropriate ways. The compiler can do this in a straightforward, if inefficient, way by using the working storages. Thus, an unedited compiled version of J77 might look like this:

J77	J51	90
90	11W1	
	J60	
	20W1	
	7091	92
92	12W1	
	11W0	
	J2	
	7090	93
91	J31	0
93	J31	0 .

The same flow diagram would be used in the compilation of J62: LOCATE ON (1) AN X SUCH THAT C(X)=(0). In fact, this routine is identical with the one just discussed, except that it requires 11W1 before the exit. It should be observed that the indefinite article "an" plays the same role in the DSCN of J62 as the quantifier "there is a" in J77. The compiler, therefore, would be provided with the knowledge that the above flow diagram, using J60 in the second segment, is the appropriate means for translating this quantifier.

Declarative and interrogative sentences in a DSCN correspond to tests in the compiled routine. Thus, the

phrase, "such that $C(X)=(0)$," leads to the question,
"Does $C(X)$ equal (0) ?" and thence to the test $J2(C(X),(0))$.

PART II

GENERAL IMPLICATIONS FOR REPRESENTATIONS

Our examination of flow diagrams has already led us to consider how some syntactical devices of English (e.g., the definite and indefinite articles) are to be rendered in the compiler. In this part, we raise at a more general level the question of the syntactical flexibility and range of the compiler languages. In Sec. VI, we ask what forms of English expressions are handled naturally and simply by the DSCN and SDSC languages, respectively. In Sec. VII, we ask how we would go about formalizing the notion of representation so that a problem-solving compiler could be given the task of designing its own representations.

VI. LANGUAGE AND REPRESENTATIONS IN THE COMPILER

We can ask appropriately about any compiler, "What range of source statements can it accept?" In Part I, we have discussed two kinds of source statements, DSCN's and SDSC's. Let us now consider in a little more detail the range of English-language expressions that these compilers can handle. We take up the language of descriptive names and the language of state descriptions in turn.

DESCRIPTIVE NAMES

The DSCN's are particularly interesting because they take the form of English sentences--imperatives, or, as we have just shown, declaratives and interrogatives. How restricted is the language of DSCN's in relation to the whole class of grammatical English sentences?

Consider the sentence, "What is the color of that apple?" The answer might be, "It's color is red," or even, "It is red." In the original sentence, "that apple" denotes a particular object; "the color," an attribute of that object; and "what," the unknown value of the attribute. In the replies, "it" denotes the same object as "that apple"; "red," the (now known) value of the attribute. Thus, we might represent the question and the first answer, respectively, as:

? = Color (that apple)

Red = Color (it).

The second answer can be interpreted as synonymous with the first if we stipulate that "red" can be a value only of the function "color."

The process in IPL-V that provides the answer to such questions is J10: FIND THE VALUE OF ATTRIBUTE (0) OF (1). In terms of our example, this is: "Find the value of attribute 'color' of that apple." Thus, the process that answers the question takes two inputs--the names of the attribute and the object--and produces the value as its output. It defines a function, in the mathematical sense of the term. In the statement of the process, "attribute" is in apposition with "color," the former term specifying the genus to which the argument belongs. We could equally (though not quite grammatically) have said, "of the object, 'that apple.'" Likewise, the phrase, "Find the value of the color," is synonymous with, "Find the color," "value" being in apposition with the (implicit) "?". That is, in English, we abridge, "The value of the attribute, the color, of the object, that apple, is red," to, "The color of that object is red," or, "That object is red." We can do this because "color" is an attribute, "that apple" is an object, and "red" is a value--nothing is added to meaning by making these classifications explicit.

This example shows how, in general, we can handle a wide class of grammatical forms within the framework of the DSCN's. Interrogatives are unknowns, like the x 's of algebra, whose genera may be specified, in part or full, to identify them. The couple "the ... of ..." signals a determiner--a phrase that names something by giving enough of its properties to tag it uniquely. Pronouns (e.g., "it") and pronominal adjectives ("that") identify by reference to terms that have occurred in previous sentences. "Find" is a general process that replaces a determiner by the object determined. Appositive phrases and relative clauses provide additional identificatory information about the object to which they refer. Adjectives, adverbs, prepositional phrases (other than "of" phrases), and adjectival nouns have the same function--identification or description. Quantifiers ("there is," "all," "a," "some," etc.) require special treatment--several of them have already been discussed.

The present DSCN Compiler was constructed specifically to handle verbs (processes), determiners (especially those involving "the ... of ..."), and proper names (in IPL-V these are always locations). Essentially, what the compiler does is replace determiners by the appropriate proper names, using the FIND processes for the compilation. In the previous section, we indicated how loops and flow diagrams could be used to handle quantifiers and con-

junctions, including "and," "but," and "if ... then." Pronouns could be handled in a manner similar to that used for determiners. Appositive and modifying words, phrases, and clauses could be used as aids in identifying proper names. It appears that with these extensions, the DSCN format would encompass most of the forms of grammatical English sentences. Only programming, of course, can determine to what extent this claim is correct.

STATE DESCRIPTIONS

Just as the DSCN language admits of considerable flexibility in representing English sentences, so the SDSC language admits of broad flexibility in the representations of information in the computer. This flexibility is achieved by using description lists as the holders of information. Each computer address that is referred to in the SDSC is represented by a cell having a description list. On the description list are the attributes NAME, TYPE, P, Q, SIGN, SYMBOL, LINK--i.e., precisely the attributes that name the fields in an IPL word (and the name of that word) and that appear on the coding sheet. Since the SDSC makes these attributes explicit, the program that uses the SDSC's need not be provided with this information in any format more specialized than the description list format itself. Moreover,

additional attributes and their values can be added, ad lib, to the description list.

By introducing attributes that refer to a particular machine representation of IPL, the SDSC language is readily extended to admit statements about the relation between IPL and its particular representations. For example, suppose the fields in the words of a particular computer were designated by the attributes DECREMENT, ADDRESS, etc. We could then define a machine language representation of IPL by setting up appropriate correspondences between IPL attributes and machine language attributes. (For example, we might specify SYMB = DECREMENT, LINK = ADDRESS, etc.) We shall indicate in the next section how this technique can be used to give the Heuristic Compiler the capability of designing appropriate representations--hence, how the compiler itself might choose an appropriate machine language representation of IPL prior to undertaking the task of compiling IPL into a machine language program.

There is also no necessity in SDSC that each description list should give the description of the contents of a single computer address. Alternatively, it may describe a whole list. Suppose, for example, we wish to represent the fact that the symbol in cell H5 is J3, and that H5 is linked to a pushdown list having unknown con-

tents. All of this information can be given in two description lists:

1.	NAME	SYMB	LINK	KIND OF OBJECT
	H5	J3	R0	Pushdown cell
2.	NAME			KIND OF OBJECT
	R0			List

We have already illustrated how this flexibility is used to represent segments of instructions in flow diagrams.

VII. THE DESIGN OF REPRESENTATIONS

One distinction between the restricted, relatively simple tasks we call "coding," and the broader, more difficult tasks we call "programming," is that the latter may encompass the selection or design of an appropriate problem representation, while the former do not. Our discussion of languages now enables us to see what is involved in the design or selection of a representation, and what we would need to do in order to give the Heuristic Compiler the capacity to grapple with such design and selection tasks. To illustrate this point, we shall take an example of a representation problem within the structure of IPL-V itself.

Let us suppose that we had an operating "basic" IPL system, quite like the language defined in the IPL-V Manual, except that the description list processes were omitted. We now give a programmer the task of introducing description lists into the language, using the basic system itself to define them, without writing any new machine code.

What do we mean by "introducing description lists"? We mean that we wish to be able to associate with the name of an object (which in IPL is always an address) a description of that object. The description consists of a set of pairs: one member of each pair is an attribute;

the other member of the pair is the value of that attribute for the object to which the description belongs. Moreover, we wish to be able to store and retrieve descriptive information about objects. That is, we wish to be able to add new pairs to descriptions, and when we are given the name of an object and an attribute, we wish to be able to find the value of that attribute for that object. Stated formally:

With every object A_i , we associate a set of pairs, (B_j, C_{ij}) . The number of pairs is to be arbitrary and variable, and we want a process that will answer questions of the form: $? = B_j(A_i)$.

How could a programmer solve this problem? By the basic conventions of IPL-V, "object" already means "address." Thus, he must find some way of associating a set with each address. Again, in IPL, the standard way to represent a set is by a list. The question then becomes, "What list can we associate with an address?" The basic relations that are represented in IPL-V are CONTENT OF and NEXT. The CONTENT OF a cell is the SYMB of that cell, and the NEXT of the cell is its LINK. Moreover, basic processes exist for FIND CONTENTS ($P = 1, Q = 1$) and FIND NEXT (J60). Thus, the set we associate with an address can be taken to be the list whose name is the SYMB (or, alternatively, the LINK) at that address. Let

us call this list (pursuing the first alternative) the description list associated with the address.

We must next define a format for description lists that will represent the pairing of attributes and values. One method would be to associate a pair of words with each element of the description list, again using the relations CONTENT and NEXT. Thus, if S_j is the content of the j^{th} number of the description list, we could define $B_j = \text{CONTENT OF } S_j$, and $C_{ij} = \text{CONTENT OF NEXT OF } S_j$. (This is substantially the representation that was used in an earlier version of IPL. An even simpler representation would make $C_{ij} = \text{LINK OF } S_j$.) Now, to add a pair, B_j, C_{ij} , to the description list of A_i , we add a cell to the description list, assign it a SYMB (S_j), assign B_j as the SYMB of S_j , and assign C_{ij} as the SYMB of the LINK of S_j . Similarly, if we are given A_i and B_j , to find C_{ij} , we first find the description list, CONTENT OF A_i , and we go down this list comparing B_j with the CONTENT of the CONTENT of each location on the list. When we obtain a match, we find the CONTENT of the cell next to the matched cell, and this is the desired value. These processes follow from the representation.

An alternative representation is obtained by dividing the members of the description list into two subsets--its ODDS and its EVENS. We then take the ODDS as the attributes; the value of an attribute is simply the EVEN that follows

it. These definitions, again, make use only of the relations CONTENTS and NEXT. Thus, an ODD of a list is the FIRST of the list or the NEXT of the NEXT of an ODD.

This definition allows us to construct a loop that will find, in sequence, all the attributes on the list. Given the location of an attribute, a FIND CONTENTS OF NEXT finds its value. This representation is, of course, the one actually adopted in IPL-V.

It will be instructive to see what the program for J10; FIND THE VALUE OF ATTRIBUTE (0) OF (1), looks like in each of these representations. We write the two programs side by side:

J10	J51		J10	J51	
	12W1			12W1	Find description list.
92	J60		92	J60	
	7090			7090	
	60W1			60W1	
	52H0			52H0	Find next attribute.
	12H0			11W0	
	11W0			J2	Test if it is equal to (0).
	J2			7091	
	7091			11W1	
	J60			J60	If so, find value.
	52H0	J31		52H0	J31
91	30H0		91	11W1	<u>list.</u>
	11W1	92		J60	92
90	30H0	J31	90	30H0	J31
					Proceed down description/
					Exit, attribute not
					found.

We see from this example that designing a suitable representation amounts to finding an isomorphism. A "description" was defined in terms of certain elements (objects, attributes, values), relations between elements

(e.g., the attributes of an object), and processes (e.g., FIND THE VALUE OF (0) OF (1)). The programmer had to find a set of elements, relations, and processes defined in IPL-V that were isomorphic with the required elements, relations, and processes. I have not worked out how such a search could be automated, but the main requirements are clear. In particular, to enable the Heuristic Compiler to perform this search, it would have to be provided with lists of the available elements, relations, and processes, or it would have to be able to recognize such things when they were described in the DSCN or SDSC languages. For example, it would have to recognize that every determiner (e.g., "the ... of ...") defines a relation.

As a second example of what is involved in designing a representation, let us consider the representation of IPL-V words in SDSC. Since each word consists of a number of symbols belonging to different fields, we can again use the description list format, in which we equate "field" with "attribute" and "symbol in field" with "value of attribute." But the symbols in the NAME, SYMB, and LINK fields themselves contain encoded information, for they are in the form ANNNN, where A is an alphameric symbol, and NNNN is a number. Hence, we represent each of these symbols by an object with a description list containing the attributes REGION and LOCATION. The value of REGION is the alphameric symbol A; the value of LOCATION, the

number NNNN. Therefore, the information that a "local symbol" is one with $A = 9$ can be represented. Again, the lesson here is that we must create an isomorphism between the elements of the representation, their relations, and the structure to be represented.

Suppose, as a third example, that we set ourselves the task of writing a program to sort a bridge hand. To accomplish this task the meanings of "sort" and "bridge hand" must be known. A bridge hand is a set of (13) elements, each characterized by a primary characteristic, suit (4 possible values), and a secondary characteristic, denomination (13 ordered values). Sorting means ordering a set of elements by one or more characteristics, taking account of the ordering of values where this is defined.

In this case we find a straightforward isomorphism: Each element in the bridge hand is to be represented by an object having a description list with attributes SUIT and DENOMINATION. A sorted bridge hand is to be represented by a list of such elements, with the ordering of the list to correspond with the ordering of the sort. It now becomes a straightforward (if difficult) compiling job to write a SORT routine that will produce a list with these properties. Moreover, if it is done correctly, it should be possible to write the routine in the generalized form: SORT (0) IN FORMAT (1), where (1) enumerates the

attributes, their ordering, and the orderings of the values that define the sorted object.

We shall explore this particular scheme in more detail in Sec. VIII of Part III. Perhaps enough has been said here to demonstrate that selecting or designing a representation is a problem-solving task that can be attacked with the same general kinds of heuristic techniques as other problem-solving tasks.

PART III

EXPERIMENTS WITH REPRESENTATIONS

In Part III, we propose some extensions of the Heuristic Compiler, most of which are motivated by linguistic considerations. In particular, we explore some methods for enabling the compiler to handle input statements in forms that are close to natural language.

There are a number of important respects in which natural languages differ from the usual programming languages. We shall be especially concerned with three of these differences:

1. In natural languages, the word is the most important unit of meaning. (For the moment, we do not need to distinguish among "word," "morpheme," and "idiom.") In most computer languages, the sentence (usually an imperative sentence, called an "instruction") is the basic unit of meaning. Thus, if a person understands, separately, the verb "sort," and the noun phrase "bridge hand," he can probably obey the instruction, "Sort the bridge hand." In most computer languages, a compiler would not be able to assemble, "Sort a bridge hand," from "sort" and "bridge hand," but would have to be provided with a number of specialized sort routines.

2. In natural languages, most communication makes use of sentences in the indicative or declarative mode.

In computer languages, most sentences are in the imperative mode. Computer languages are primarily languages of command, and not languages of information, description, or advice.

3. In natural languages, many alternative sentences can be phrased that "mean" about the same thing. The recipient of a natural-language communication is able to decode the communication without too much concern for details of format. In computer languages, there are various harassing constraints on format. Failure to observe these constraints usually causes an error condition.

These differences are, of course, differences of degree and not of kind. Moreover, research on computer languages over the past decade has already made substantial progress toward decreasing or erasing them. Basic processes have become more general and parameterized; various forms of declarative statements have been introduced; compilers have been designed to accept relatively informal input statements. The gap, however, between natural language and computer languages is still large, and annoying to those who are engaged in man-machine communication.

We are here concerned with extensions of the Heuristic Compiler directed towards reducing these

differences.* Section VIII indicates how generalized processes (verbs) can be introduced into the DSCN Compiler. Section IX describes subroutines for storing and retrieving descriptive information in declarative sentences. Section X describes an approach toward natural-language flexibility in input statements for the SDSC Compiler, making use of the descriptive information provided by the techniques of Sec. IX.

*The thinking reported here, particularly in Sections IX and X, was greatly stimulated by the work of Robert K. Lindsay, "The Reading Machine Problem" (Unpublished Ph.D. Thesis, Carnegie Institute of Technology, 1960), which has been revised as Toward the Development of a Machine Which Comprehends, University of Texas, Austin, May 1961. I owe a great deal also to stimulating discussions with Hugh Kelly and Allen Newell of The RAND Corporation and Carnegie Tech, respectively.

VIII. GENERALIZED PROCESSES

We shall consider two classes of processes: one designated by the verb "find," the other by the verb "sort."

"FIND" PROCESSES

The possible interpretations of FIND are numerous to the point of being meaningless. Any routine that takes some symbolic structures as inputs and produces one or more other structures as outputs may be called a process for "finding" the latter. Thus: FIND SINE A, FIND THE STATE DESCRIPTION OF ROUTINE K, FIND THE PROOF OF THEOREM T. From this point of view, a program like the General Problem-Solver is simply a fairly general FIND routine.

Hence, any routine flexible enough to interpret correctly the verb "find" wherever it occurs in normal English prose, would have to make considerable use of context. In the present section we shall aim at a lesser degree of flexibility. Consider the two classes of processes typified by:

FIND THE state description OF routine K, and
FIND A ON

The first example designates an object associated in a particular way with a specific object, K. In IPL-V there is provided a special format, the description list, for holding such information in memory, and a set of

processes, J10 to J16, for entering and retrieving the information. Section IX examines definite descriptions of this special kind at length. In the present section, then, we shall limit ourselves to the verb "find" as it occurs in instructions like, FIND A ... ON ..., and FIND THE ... ON Even this scheme covers a considerable variety of processes:

FIND A symbol, S10, ON list L.

FIND AN object whose type is A4 ON list structure L.

FIND THE third symbol ON list L.

FIND THE largest integer data term ON the lists
of list L.

In the first two examples, the indefinite article indicates that the object sought is not necessarily unique; in the last two examples it is. In the first two examples the properties that define the object sought are absolute--their presence or absence can be ascertained without reference to other objects. In the last two examples, the properties are relative; and indeed, in the fourth example, the entire set of objects must be examined before the one sought can be identified. In the first example, the object sought is designated by proper name (and the information is added that it is a "symbol"). This example can be approximated to the others by rephrasing it: FIND A token equal to S10 ON list L.

With these preliminaries out of the way, we shall describe in detail a rather general FIND process. The object sought will be specified by some sublist of the symbols on the pushdown list of the communication cell, H0. The specification of the place to be searched will be given by other symbols on that pushdown list. Hence, we can symbolize the desired process:

FIND A $F_1[(0), (1), \dots, (k)]$ IN $F_2[(k+1), (k+2), \dots, (n)]$,

where (0), (1), etc., designate, as usual, the symbols on the pushdown list of H0.

We suppose that the compiler is provided with a lexicon that contains, among others, the words "FIND," " F_1 []," and " F_2 []." The lexical entry for each of these is a description list containing, for "FIND" and " F_1 ," the attribute IPL ROUTINE, and for " F_2 ," the attribute TYPE OF OBJECT.

The IPL ROUTINE associated with FIND in the lexicon will contain certain variables, to be replaced by constants derived from an examination of F_1 and F_2 . We will first present the routine, as it would appear in the lexicon, and then explain the meaning of certain of the symbols in it.

FIND	J5n 11W(n-k) 1090 GEN (F_1, F_2) J5	J3n
90	40H0 11Wk TEST(F_1) J5 7090	J8

As previously indicated, k is the number of arguments in F_1 , (n-k) the number of arguments in F_2 , and n the number of arguments in the two functions taken together. Thus J5n preserves W0 to Wn and moves all the arguments of F_1 and F_2 into the W's; J3n restores W0 to Wn. The instruction 11W(n-k) brings into H0 the arguments of F_2 , while 11Wk brings in the arguments of F_1 . The numbers k and n-k are to be determined, of course, by examining F_1 and F_2 . The variable, GEN(F_1, F_2), is to be replaced with a generator obtained from the lexical entry for F_2 , while TEST(F_1) is to be replaced with a test associated with the lexical entry for F_1 . All the other symbols have their usual IPL-V meanings.

We can now compile FIND A F_1 IN F_2 as follows:

1. Get IPL ROUTINE of FIND.
2. Supply values for k, n, and n-k where required.
3. Make list of subroutines required (GEN(F_1, F_2), TEST(F_1)).
4. Get the IPL ROUTINE of TEST, supply it with its arguments, and insert it in the IPL-V code for FIND.
5. Get TYPE OF OBJECT of F_2 , and get the associated GEN for that type in the dictionary of generators; insert it in the IPL-V code for FIND.

A word may be added to this account to indicate how relative properties are handled in FIND THE THIRD ..., or FIND THE LARGEST. In these cases, the test needs to be based on a recursive process--in the case of FIND THE THIRD ..., a counting process; in the case of FIND THE

LARGEST ..., a process that resets LARGEST SO FAR equal to the larger of LARGEST SO FAR and the current integer. The test--the function F_1 --would, in these cases, make provision for storing in the W's the intermediate products of calculation.

"SORT" PROCESSES

As our second essay toward generalized processes, we take the verb "sort." Suppose that the objects we wish to sort are description lists. It is easy to construct a general routine, SORT (0) ON (1), where (0) is the list of objects to be sorted and (1) is a specification of the attributes, their ordering, and the orderings of attribute values on which the sorting is to be based. Thus, if (0) were a list of description lists representing the cards in a bridge hand, (1) would specify that the attributes are suit and denomination (in that order), that the suit values are S,H,D,C, in that order, and the denomination values A,K,Q,J,10,9, etc., in that order. A further step toward generalization would allow the sort routine to be compiled from a definition of the collection of objects to be sorted. Let us see how this can be done when the object to be sorted is a bridge hand. First, we store in memory description lists providing information about the terms LIST, DESCRIPTION LIST, and ATTRIBUTE:

LIST
 TYPE: Class of objects
 ATTRIBUTES: Type of members

DESCRIPTION LIST
 TYPE: Class of objects
 ATTRIBUTES: Attributes

ATTRIBUTE
 TYPE: Class of symbols
 ATTRIBUTES: Values.

That is to say, a list is a class of objects; the description list for any class of objects of type LIST will have the attribute CLASS OF MEMBERS. A description list is also a class of objects; the description list for any class of objects of type DESCRIPTION LIST will have the attribute ATTRIBUTES. An attribute is a class of symbols; the description list for any class of objects of type ATTRIBUTE will have the attributive VALUES.

Next, we store in memory description lists providing information about the terms BRIDGE HAND, CARD, SUIT, and DENOMINATION.

BRIDGE HAND
 TYPE: List
 MEMBERS: Cards

CARD
 TYPE: Description List
 ATTRIBUTES: Suit, denomination

SUIT
 TYPE: Attribute
 VALUES: Spade, hearts, diamonds, clubs

DENOMINATION
 TYPE: Attribute
 VALUES: A,K,Q,J,10,9,8,7,6,5,4,3,2

Now we can compile SORT THE BRIDGE HAND (0), as follows: From the information just stored, we find that BRIDGE HAND has the type LIST. From LIST, we find that BRIDGE HAND will have the attribute TYPE OF MEMBERS. Finding the value of this attribute, we determine that the members of bridge hands are cards. CARD is a description list, which has the attribute ATTRIBUTES. The attributes of cards are suit and denomination. We would therefore compile the sort routine to sort on suit and denomination, in that order of priority. Examining the values of these two terms, in turn, we find the order in which these values are to be arranged in sorting.

If it were known to the program that a bridge hand is a list of description lists, then the sorting routine could obtain the information about attributes and values by direct examination of one or more examples of a sorted bridge hand, and without being given the information about CARD, SUIT, or DENOMINATION explicitly. If the examples were not too special (e.g., a hand of thirteen spades), the program could determine what attributes a card possessed, which of these was relevant to the sorting, the hierarchy of attributes, and the ordering, if any, of the values of each. This information could then be used to compile the specific sorting routine required.

Thus, we see that the key to providing a generalized routine for a verb like SORT lies in providing syntactically or semantically the information needed to supply the routine with the parameter values it requires. This can be accomplished (syntactically) through a scheme of declarative sentences that describe the objects under consideration; or (semantically) by providing examples that can be analyzed. Moreover, the description itself can be generated inductively from examples.

The description that would allow compilation of the "sort" routine could also be used to compile TEST IF X IS A BRIDGE HAND. Thus, storing descriptions of classes of objects is an important means for factoring sentence meanings of nouns and verbs, respectively. It provides a powerful basis for introducing general processes.

RECURSIVE FUNCTIONAL LANGUAGES

Consider an instruction language consisting of a set of functions each admitting as arguments the values of functions of the set. Each of the functions can be regarded as a FIND instruction--i.e., FIND THE VALUE OF F FOR THE GIVEN ARGUMENT VALUES. It may be executed recursively by finding, first, the values of each of its arguments, then using these to compute the value of the function. Hence, the interpreter of such a language may itself be regarded as a generalized FIND instruction.

Next, consider an instruction like SORT OBJECT A, in a recursive functional language. The definition of SORT, if it is a generalized routine like those described in the previous section, may read something like ARRANGE ACCORDING TO THE ATTRIBUTE VALUES OF. The definition has different arguments from SORT itself--it refers to the attribute values of A instead of the object A. The interpreter would need to be general enough to replace SORT A by ARRANGE A BY THE ATTRIBUTE VALUES OF A; then execute FIND THE ATTRIBUTE VALUES OF A, and insert the value of this function as the second argument of the ARRANGE function. We have already indicated how the FIND might be accomplished.

With a little further generalization, the scheme could handle apposition--e.g., SORT THE BRIDGE HAND, A. The phrase in apposition would provide information about the type of the object designated, and as we have seen, this information could be used to find the other argument of the ARRANGE routine. Further light will be cast on apposition in the next section, where we shall discuss modifiers that identify an argument, and their relation to modifiers that describe the object.

IX. DEFINITE DESCRIPTIONS

The meaning of much descriptive and expository prose can be captured in a fairly simple language--a sub-language of English--that uses only the verb "is" and noun phrases with definite or indefinite articles in subjects and predicates. Consider the following example, which, while it does not fit this restricted form exactly, is not far from it:

The state description of a routine consists of a list of affected cells. For each affected cell on the list, the state description specifies its input state and its output state.

SYNTACTIC CHARACTERIZATION

We might proceed to formalize this description in either of several ways. I shall call the first of these syntactical, since it makes statements about the terms "state description," "affected cells," "input state," and "output state." These statements can then be stored in association with the relevant terms in a lexicon.

The type of "state description" is "description list";
The attribute of "state description" is "list of affected cells";
The type of "list of affected cells" is "list";
The type of "affected cell" is "description list";
The attributes of "affected cell" are "input state" and "output state."

We have already seen this kind of description in our discussion of generalized SORT routines in the previous

section. Readers who are familiar with the notation known as Backus normal form will observe that a syntactical description of this kind could without much difficulty be translated into that form--or a slight extension thereof. Our interest, however, is in staying close to natural English.

SEMANTIC CHARACTERIZATION

An alternative, semantic, formalization characterizes a given type of object (STATE DESCRIPTION in this instance) by describing an example:

X1 is a state description of a routine if
 there are an X0, X2, X3, X4,
 and X5, such that:
X0 is a routine;
X1 is the state description of X0;
X2 is the list of affected cells of X1;
X3 is a member of X2;
X4 is the input state of X3;
X5 is the output state of X3.

Properly interpreted, the example implies the syntactic description we gave previously. Consider, for instance, "X3 is a member of X2." With the convention that only lists have members, this statement implies that X2 is a list. From the previous statement, "X2 is the list of affected cells of X1," we observe that this list is the value of the attribute, LIST OF AFFECTED CELLS, of X1.

We can store the example in memory by storing a description list, X0, with attribute STATE DESCRIPTION

having value X1. X1, in turn, is a description list, with attribute LIST OF AFFECTED CELLS having the value X2. X2 is a list whose sole member is X3. X3 is a description list with attribute INPUT STATE having value X4, and attribute OUTPUT STATE having value X5.

ANNEXING DESCRIPTIVE INFORMATION TO AN INFORMATION STORE

Let us use symbols from the X region--e.g., X114, X33--to designate nouns. These nouns will be either proper names (of objects to be represented by lists or list structures) or attributes. Consider now the sentence:

"X114 is the X33 of the X25 of X105."

In this sentence, which is grammatical if inelegant English, "X114" and "X105" are proper names, while "X33" and "X25" name attributes. The objects referred to in this sentence are X114 (or synonymously, the X33 of the X25 of X105), the X25 or X105, and X105. The problem of annexing the information provided by this sentence to an existing memory store depends on what is already in the store.

Suppose, as a first possibility, that no information has been stored previously about the objects mentioned in this sentence. We store the new information by creating a name, call it X200, and assigning it as the value of attribute X25 of X105. Then we assign X114 as the value of attribute X33 of the newly named object, X200.

Suppose, however, that we had previously stored in memory the information that an object named X130 was the value of attribute X25 of X105. Then, to store the new information, we would first have to find X130, and then assign X114 as the value of attribute X33 of X130.

In the first case, we annexed the new information by two ASSIGN processes--in IPL-V, two applications of J11. In the second case, we annexed the new information by a FIND process (J10 in IPL-V) followed by an ASSIGN. We can write a general routine to accomplish this. In processing a sentence like the one we are using as example, we start at the extreme right and search in memory for the object named. If we find it, we proceed to the left, find the first attribute, and find the value of this attribute of the object. If the value exists, it becomes a new object on which we can repeat the process, moving to the next attribute to the left.

When we fail to find an object meeting the description (when J10 fails), we enter a second phase. We now proceed from the left-hand side of the sentence, creating names for new objects as these are needed, and annexing their descriptions to them (by J11), until we reach an object that is already mentioned in memory.

Thus, depending on what is already stored in memory, the same piece of information in the input sentence can

serve either as a descriptive phrase, providing new information to be annexed to the memory structure, or as an identifying phrase, to be used in locating the place in memory where the new information is to be annexed.

In this scheme ambiguity is entirely possible. It can enter because the scheme allows indefinite, as well as definite description. Again, an example will make the point clear. We consider the following sequence of four input sentences:

X114 is the X33 of the X25 of X105.
X115 and X116 are the X99 of the X34 of the
X25 of X105.
X125 is the X41 of a member of the X71 of the
X24 of X105.
X117 is the X75 of the member whose X41 is
X125 of the X71 of the X24 of X105.

Suppose we begin with no information about X105 in memory. Then, as we have seen, the first sentence is stored by two executions of J11. The first creates a new object, say X200, and assigns X114 as its X33; the second assigns X200 as the X25 of X105.

The second sentence is stored by two executions of J11 and one of J10. Working from the right, by J10, X200 is found to be already in memory as the X25 of X105. But there is no value for attribute X34 of X200. Hence, a list, say X201, is created whose members are X115 and X116, and X201 is assigned as the value of X99 of another new object, say X202. Finally, X202 is assigned as the X34 of X200.

Storing the third sentence brings about the creation of X203, whose X41 is X125; of a list X204, of which X203 is member; and of an object, X205, whose X71 is X204, and which is, in turn, the X24 of X105.

The fourth sentence introduces a new complication. It refers to "the member of the X71 of the X24 of X105 whose X41 is X125." A series of J10's will find the X71 of the X24 of X105--that is, X204. The sentence now calls for locating that member of X204 whose X41 is X125. From the previous paragraph, we see that the object in question is none other than X203. Thus X117 would be assigned as the X75 of X203.

Now let us return to the question of ambiguity. Suppose that the fourth sentence read:

X117 is the X75 of the member whose X76 is
X130 of the X71 of the X24 of X105.

Now, when we examine X204, we find, as before, that it has a member, X203. But we have no information to tell us whether or not X203 is "the member of X204 whose X76 is X130." Hence, we do not know whether to assign X117 as the X75 of X203 or to create a new member of the list X204--say X205, and assign X117 as the X75 of X205.

The ambiguity is not a consequence of the particular annexing scheme we have used, but resides more deeply in the nature of things. Let r and s be relations; A , B , C , and X , objects. Suppose we know that ArB and that there

exists a Y such that CsY and YrB. Then we can neither affirm nor deny that Y is identical with A. (The same difficulty arises in R. Lindsay's program for annexing genealogical information to a family tree.* If we know only that Isaac is a son of Abraham, and Jacob is a son of a son of Abraham, then we do not know whether or not Jacob is a son of Isaac.) Our only recourse is to arrange the annexing routine so that, when it detects such an ambiguity, it outputs an appropriate question.

A program for an annexing routine is given in detail in Appendix B. It will not deal with the ambiguity problem just discussed, but is in all other respects capable of annexing to memory the contents of sets of sentences of the kinds we have been considering.

*Op. cit., p. 43.

X. COMPILATION OF ROUTINES FROM PARTIAL DESCRIPTIONS

One part of the flexibility of natural language depends on the problem-solving capacity of the listener. Information already in his memory allows him to supply details that are omitted from, or only implicit in, the communication. Suppose we wish to give a (human) programmer the task of coding a routine in IPL-V for J3. We might tell him:

"Write a routine, J3, that changes the contents of cell H5 from [S1,R0] to [J3,R0]."

The programmer, familiar with IPL-V, knows that H5 is a so-called "pushdown cell," and that what is wanted is to replace the contents of the SYMB field of that cell, whatever they may be, by the symbol "J3."

What instruction would we have to give to the SDSC Compiler to induce it to perform the same task? The instructions would read:

"Compile the IPL-V definition of the following routine: The NAME of the routine is J3. The LIST OF AFFECTED CELLS of its STATE DESCRIPTION has a member, whose NAME is H5, whose INPUT STATE is the list, S1, R0, and whose OUTPUT STATE is the list, J3, R0."

Given some such statement, the annexing routine described in the previous section could construct an appropriate description list as input to the SDSC Compiler. In Part I, we showed how the SDSC Compiler could then write the desired routine from its state description.

Now the instructions to the compiler have required about three times as many words as the instructions to the human programmer. The reason is easy to see. The essential information to be provided is that the cell to be changed is H5, and that the change is to replace an unknown symbol by the symbol J3. The compiler cannot receive this information without explicit mention that J3 is the first symbol in the list of the output state of affected cell H5 of the list of affected cells of the state description of the routine named J3. The human programmer is capable of supplying this additional information, because he knows what the structure of a state description is like.

The last observation suggests that we might give the compiler the same capability by providing it prior information about the structure of a state description, and allowing it to fill in the implicit detail. Let us employ the language of Sec. IX to see how this might be done.

The complete description list, necessary to the Heuristic Compiler, is equivalent to the following set of sentences:

Let X100 be a routine whose NAME is J3.
Let X101 be the STATE DESCRIPTION OF X100.
Let X102 be a member, whose NAME is H5, of
the LIST OF AFFECTED CELLS OF X101.
Let S1, R0 be the INPUT STATE OF X102.
Let J3, R0 be the OUTPUT STATE OF X102.

In parallel fashion, we can store in memory a "template" for the description of a routine. We will use symbols from region "Y" to denote variables.

Y1 is the NAME of Y2.
Y3 is the STATE OF DESCRIPTION of Y2.
Y4 is the LIST OF AFFECTED CELLS OF Y3.
Y5 is a typical member of Y4.
Y6 is the NAME of Y5.
Y7 is the INPUT STATE OF Y5.
Y8 is the OUTPUT STATE OF Y5.
X7 and Y8 are lists.

Now to fit the specific example to the template, we identify J3 with Y1, X100 with Y2, X101 with Y3, X102 with Y5, and so on. Suppose, however, the example were incomplete, as follows:

X100 is a routine whose NAME is J3.
N101 is a cell whose NAME is H5.
S1, R0 is the M1 of X101.
J3, R0 is the M2 of X101.

Here "M1" and "M2" designate attributes whose meanings are not given in the lexicon. It should not be too difficult to devise a process for matching this description with the template. The matching process would discover that X101 has to be identified with Y5, and X100 with Y1. New objects could then be created to correspond with Y3 and Y4, and the appropriate description list stored in memory. The SDSC Compiler, taking this description list as its input, could now compile the code for J3.

Our basic proposal, then, for compiling routines from partial descriptions rests on two devices: (1) the

use of an annexing routine like that described in Sec. IX as a means for constructing description lists from expository sentences; (2) the use of templates to provide information that is not given explicitly in the input sentences. The routine for accomplishing the second task has not yet been written.